

Frontend Developer Interview Knowledge Base

AI Interview Coach  RAG Knowledge Base

2026-07-11

Contents

Frontend Developer Interview Knowledge Base	2
HTML	2
Semantic HTML	2
Forms	3
Tables	3
Media	3
Accessibility (HTML)	4
CSS	4
Selectors	4
Box Model	4
Positioning	5
Flexbox	5
CSS Grid	6
Responsive Design	6
Media Queries	7
Animations	7
JavaScript	7
Variables	7
Scope	8
Hoisting	8
Closures	9
Promises	9
Async/Await	10
Callbacks	10
Event Loop	10
DOM	11
Events	11
ES6+ Features	12
Modules	12
React	13
Components	13
Props	13
State	13
Hooks	13
useState	14
useEffect	14
Routing	14
Context API	15

Lifecycle	15
Performance Optimization	16
Browser & Web APIs	16
Local Storage	16
Session Storage	16
Cookies	17
CORS	17
HTTP Methods	17
Best Practices Summary	18
Performance	18
Accessibility (General)	18
Common Interview Question Concepts	19
Common Mistakes (Cross-Cutting)	19

Frontend Developer Interview Knowledge Base

Concise, retrieval-friendly reference for an AI Interview Coach. Each topic is self-contained with definitions, examples, best practices, and common mistakes.

HTML

Semantic HTML

Definition: HTML elements that convey meaning about their content/structure, not just presentation (e.g., <header>, <nav>, <main>, <article>, <section>, <footer>, <aside>) versus generic containers (<div>,).

```
<header>Site Header</header>
<nav><a href="/">Home</a></nav>
<main>
  <article>
    <h1>Blog Post</h1>
    <section>Content...</section>
  </article>
  <aside>Related links</aside>
</main>
<footer>© 2026</footer>
```

Best Practices: Use semantic tags for structure and SEO; reserve <div>/ for styling hooks with no inherent meaning; one <h1> per page, logical heading order.

Common Mistakes: Using <div> for everything (“divitis”); skipping heading levels (<h1> → <h3>); using /<i> instead of / for semantic emphasis.

Interview Tip: Be ready to explain *why* semantics matter: accessibility (screen readers), SEO, and maintainability.

Forms

Definition: <form> collects user input via controls (<input>, <select>, <textarea>, <button>) and submits it to a server.

```
<form action="/submit" method="POST">
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>
  <input type="password" name="password" minlength="8" required>
  <button type="submit">Submit</button>
</form>
```

Best Practices: Always pair <label> with for/id (or nest input inside label) for accessibility; use correct input types (email, number, tel) for built-in validation and mobile keyboards; use required, pattern, min/max for native validation before adding JS validation.

Common Mistakes: Missing <label> (bad for screen readers); relying only on client-side validation (must also validate server-side); using <div onclick> instead of <button> (loses keyboard accessibility).

Interview Tip: Know GET vs POST in forms — GET appends data to URL (visible, cacheable, idempotent), POST sends data in body (not cached, used for mutations).

Tables

Definition: <table> displays tabular data using <thead>, <tbody>, <tr>, <th>, <td>.

```
<table>
  <thead>
    <tr><th scope="col">Name</th><th scope="col">Age</th></tr>
  </thead>
  <tbody>
    <tr><td>Alice</td><td>30</td></tr>
  </tbody>
</table>
```

Best Practices: Use tables only for tabular data, never for page layout; use scope on <th> for accessibility; use <caption> to describe the table.

Common Mistakes: Using tables for layout (legacy anti-pattern — use Flexbox/Grid instead); missing <th> headers, breaking screen-reader navigation.

Media

Definition: , <video>, <audio>, and <picture> embed media content.

```


<picture>
  <source srcset="cat.webp" type="image/webp">
  
</picture>

<video controls width="600">
  <source src="movie.mp4" type="video/mp4">
</video>
```

Best Practices: Always provide meaningful alt text (empty alt="" for decorative images); specify width/height to prevent layout shift (CLS); use loading="lazy" for offscreen images; use <picture>/srcset for responsive/art-directed images.

Common Mistakes: Missing alt attributes; using large unoptimized images; forgetting controls on video/audio for accessibility.

Accessibility (HTML)

Definition: Building HTML that is usable by everyone, including people using assistive technology (screen readers, keyboard-only navigation).

Key practices: semantic HTML first; ARIA roles/attributes (aria-label, aria-hidden, role) only when native HTML can't express the semantics; sufficient color contrast; visible focus states; logical tab order.

```
<button aria-label="Close dialog" onclick="closeModal()">x</button>
<div role="alert">Form submitted successfully!</div>
```

Best Practices: “No ARIA is better than bad ARIA” — prefer native elements (<button> over <div role="button">). Test with keyboard-only navigation (Tab, Enter, Esc).

Common Mistakes: Using div/span with click handlers instead of <button>; removing focus outlines (outline: none) without a replacement; relying solely on color to convey information.

Interview Tip: Know WCAG basics — POUR principles (Perceivable, Operable, Understandable, Robust).

CSS

Selectors

Definition: Patterns used to target HTML elements for styling.

```
.card { }           /* class */
#header { }         /* id */
div p { }           /* descendant */
div > p { }         /* direct child */
a:hover { }         /* pseudo-class */
p::first-line { }   /* pseudo-element */
input[type="text"] { } /* attribute selector */
```

Specificity (low to high): universal (*) → element/pseudo-element → class/attribute/pseudo-class → ID → inline style → !important.

Best Practices: Prefer classes over IDs for styling (lower specificity, reusable); avoid !important; keep selectors shallow for maintainability and performance.

Common Mistakes: Overusing IDs for styling (specificity wars); deeply nested selectors that are fragile and hard to override.

Box Model

Definition: Every element is a rectangular box composed of **content** → **padding** → **border** → **margin**.

```
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
  box-sizing: border-box; /* width includes padding+border */
}
```

content-box (default): width/height apply to content only; padding/border add to total size.
border-box: width/height include padding and border — total size stays fixed. Widely recommended as default.

Best Practice: Apply globally: `* { box-sizing: border-box; }` to make sizing predictable.

Common Mistake: Forgetting `box-sizing: border-box`, causing unexpected overflow when adding padding/border to fixed-width elements.

Positioning

Definition: The position property controls how an element is placed in the document flow.

Value	Behavior
static	default, normal flow
relative	offset relative to its normal position, still occupies original space
absolute	removed from flow, positioned relative to nearest positioned ancestor
fixed	positioned relative to viewport, stays on scroll
sticky	toggles between relative and fixed based on scroll position

```
.parent { position: relative; }
.child { position: absolute; top: 0; right: 0; }
.navbar { position: sticky; top: 0; }
```

Best Practice: Use sticky for headers that stick after scrolling past a point; ensure absolute children have a relative/absolute positioned ancestor to avoid positioning relative to `<body>` unexpectedly.

Common Mistake: Forgetting to set a positioned ancestor, causing absolute elements to jump to unexpected locations.

Flexbox

Definition: A one-dimensional layout model (`display: flex`) for distributing space along a row or column.

```
.container {
  display: flex;
  justify-content: space-between; /* main axis */
  align-items: center;           /* cross axis */
  gap: 16px;
  flex-wrap: wrap;
}
```

```
}  
.item { flex: 1; } /* grow, shrink, basis shorthand */
```

Key properties: flex-direction, justify-content, align-items, align-content, flex-grow/flex-shrink/flex-basis, gap.

Best Practice: Use gap instead of margins for spacing between flex items (cleaner, no edge-case margin collapsing).

Common Mistake: Confusing main-axis (justify-content) vs cross-axis (align-items) properties — they swap meaning when flex-direction: column is used.

Interview Tip: Flexbox = 1D (row or column); Grid = 2D (rows and columns simultaneously).

CSS Grid

Definition: A two-dimensional layout system (display: grid) for arranging items into rows and columns.

```
.grid {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-template-rows: auto 1fr auto;  
  gap: 20px;  
}  
.item { grid-column: span 2; }  
  
/* Named areas */  
.layout {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "sidebar content"  
    "footer footer";  
}  
.header { grid-area: header; }
```

Best Practice: Use fr units for flexible tracks, repeat() and minmax() for responsive grids (repeat(auto-fit, minmax(200px, 1fr))) without media queries.

Common Mistake: Using Grid for simple 1D layouts where Flexbox is simpler; forgetting grid-template-areas names must match exactly and form a rectangular shape.

Responsive Design

Definition: Designing layouts that adapt to different screen sizes/devices using fluid grids, flexible images, and media queries.

Best Practices: Mobile-first approach (base styles for mobile, add complexity via min-width media queries); use relative units (% , rem, em, vw/vh) over fixed px; use max-width: 100% on images.

```
img { max-width: 100%; height: auto; }  
.container { width: 100%; max-width: 1200px; margin: 0 auto; }
```

Common Mistake: Designing desktop-first then retrofitting mobile styles (leads to more overrides); using fixed pixel widths that break on smaller viewports.

Media Queries

Definition: CSS rules applied conditionally based on device characteristics (width, orientation, resolution).

```
/* Mobile-first: base styles apply to all, then override for larger screens */
.container { display: block; }

@media (min-width: 768px) {
  .container { display: flex; }
}

@media (max-width: 480px) {
  .sidebar { display: none; }
}
```

Best Practice: Use min-width (mobile-first) rather than max-width (desktop-first) for cleaner cascading overrides. Common breakpoints: 480px (mobile), 768px (tablet), 1024px (desktop) — but base them on content, not devices.

Common Mistake: Mixing min-width and max-width inconsistently, creating overlapping/conflicting rules.

Animations

Definition: CSS transition animates property changes over time in response to state change; @keyframes + animation define multi-step animations independent of triggers.

```
.button {
  transition: background-color 0.3s ease, transform 0.2s;
}
.button:hover { background-color: blue; transform: scale(1.05); }

@keyframes spin {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}
.loader { animation: spin 1s linear infinite; }
```

Best Practice: Animate transform and opacity for best performance (GPU-accelerated, no layout reflow); avoid animating width/height/top/left (triggers layout/reflow).

Common Mistake: Animating expensive properties (width, box-shadow, top) causing jank; forgetting will-change hints are for rare, expensive cases only (overuse hurts performance).

JavaScript

Variables

Definition: var, let, const declare variables with different scoping and mutability rules.

```
var x = 1;    // function-scoped, hoisted, re-declarable
let y = 2;    // block-scoped, not re-declarable in same scope
const z = 3;  // block-scoped, cannot be reassigned (object contents still mutable)

const obj = { a: 1 };
obj.a = 2;    // valid – mutating contents, not reassigning reference
```

Best Practice: Default to `const`; use `let` only when reassignment is needed; avoid `var` in modern code.

Common Mistake: Believing `const` makes objects immutable — it only prevents reassignment of the binding, not mutation of object/array contents.

Scope

Definition: The region of code where a variable is accessible — **global**, **function**, and **block** scope (`let/const` introduce block scope with `{}`).

```
function outer() {
  let a = 10;
  if (true) {
    let b = 20;    // block-scoped, not visible outside if
    var c = 30;    // function-scoped, visible in whole function
  }
  console.log(a, c); // works
  // console.log(b); // ReferenceError
}
```

Interview Tip: Explain lexical scoping — a function's scope is determined by where it's *written*, not where it's *called*.

Hoisting

Definition: JavaScript moves variable and function *declarations* to the top of their scope during compilation, before execution.

```
console.log(a); // undefined (var hoisted, initialized to undefined)
var a = 5;

console.log(b); // ReferenceError (Temporal Dead Zone)
let b = 10;

greet();        // works – function declarations fully hoisted
function greet() { console.log("hi"); }
```

Key point: `var` declarations are hoisted and initialized as `undefined`; `let/const` are hoisted but remain in the **Temporal Dead Zone (TDZ)** until their declaration line executes. Function declarations are hoisted entirely (including body); function expressions/arrow functions are not.

Common Mistake: Assuming `let/const` aren't hoisted at all — they are hoisted, just not initialized, which is why accessing them early throws instead of returning `undefined`.

Closures

Definition: A closure is a function that retains access to its lexical scope (outer variables) even after the outer function has returned.

```
function counter() {  
  let count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const increment = counter();  
console.log(increment()); // 1  
console.log(increment()); // 2 (state persists between calls)
```

Use cases: data privacy/encapsulation, function factories, memoization, event handler state.

Best Practice: Use closures for private state in modules/factory functions instead of relying on global variables.

Common Mistake: Closures inside loops capturing the wrong variable with var:

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 100); // prints 3, 3, 3  
}  
// Fix: use `let` (block-scoped per iteration) → prints 0, 1, 2
```

Promises

Definition: An object representing the eventual completion (or failure) of an asynchronous operation, with states: **pending**, **fulfilled**, **rejected**.

```
const fetchData = () => new Promise((resolve, reject) => {  
  setTimeout(() => {  
    const success = true;  
    success ? resolve("Data loaded") : reject("Error");  
  }, 1000);  
});  
  
fetchData()  
  .then(result => console.log(result))  
  .catch(error => console.error(error))  
  .finally(() => console.log("Done"));  
  
// Combinators  
Promise.all([p1, p2]); // waits for all, fails fast on first rejection  
Promise.allSettled([p1, p2]); // waits for all, never rejects  
Promise.race([p1, p2]); // resolves/rejects with the first settled
```

Best Practice: Always attach a `.catch()` (or use `try/catch` with `async/await`) to avoid unhandled promise rejections.

Common Mistake: Forgetting to return a promise inside a `.then()` chain, breaking the chain's sequencing.

Async/Await

Definition: Syntactic sugar over Promises that lets asynchronous code be written in a synchronous-looking style. `async` functions always return a Promise; `await` pauses execution until the awaited Promise settles.

```
async function loadUser() {
  try {
    const response = await fetch("/api/user");
    const data = await response.json();
    return data;
  } catch (error) {
    console.error("Failed:", error);
  }
}
```

Best Practice: Use `try/catch` for error handling with `async/await`; run independent `async` calls in parallel with `Promise.all` instead of sequential `await` calls to avoid unnecessary waterfall delays.

```
// Sequential (slow):
const a = await fetchA(); const b = await fetchB();
// Parallel (fast):
const [a, b] = await Promise.all([fetchA(), fetchB()]);
```

Common Mistake: Awaiting calls sequentially when they could run in parallel; forgetting `async` functions always return a Promise, even if you return a plain value.

Callbacks

Definition: A function passed as an argument to another function, to be executed later (e.g., after an `async` operation completes).

```
function fetchData(callback) {
  setTimeout(() => callback(null, "data"), 1000);
}
fetchData((err, data) => {
  if (err) return console.error(err);
  console.log(data);
});
```

“Callback Hell”: Deeply nested callbacks that become hard to read/maintain — solved by Promises/`async-await`.

Best Practice: Prefer Promises/`async-await` over nested callbacks for readability and error handling.

Common Mistake: Not handling errors in callback-style APIs (Node.js convention: error-first callbacks `(err, data) => {}`).

Event Loop

Definition: The mechanism that allows JavaScript (single-threaded) to perform non-blocking asynchronous operations by offloading tasks and processing a queue of callbacks.

Model: Call Stack → Web APIs (timers, fetch, DOM events) → Callback Queue (macrotasks) / Micro-task Queue (Promises) → Event Loop pushes queued callbacks back to the stack when it's empty.

```

console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// Output: 1, 4, 3, 2

```

Key point: Microtasks (Promise callbacks, queueMicrotask) always run before the next **macro-task** (setTimeout, setInterval, I/O), even if the timeout is 0.

Interview Tip: This exact code trace ("1,4,3,2") is one of the most common JS interview questions — practice explaining it step by step.

DOM

Definition: The Document Object Model is a tree-like, programmatic representation of an HTML document that JavaScript can read and manipulate.

```

const el = document.querySelector(".card");
el.textContent = "Updated";
el.classList.add("active");

const newEl = document.createElement("div");
newEl.innerText = "New element";
document.body.appendChild(newEl);

```

Best Practice: Batch DOM updates / use DocumentFragment to minimize reflows when inserting many elements; prefer textContent over innerHTML to avoid XSS when inserting user data.

Common Mistake: Using innerHTML with unsanitized user input (XSS vulnerability); causing layout thrashing by repeatedly reading then writing layout properties in a loop.

Events

Definition: Actions (click, input, keydown, etc.) that trigger JavaScript handlers. Events **bubble** (child → parent) by default and can be **captured** (parent → child).

```

document.querySelector("button").addEventListener("click", (e) => {
  e.preventDefault(); // stop default browser action
  e.stopPropagation(); // stop bubbling to parent
  console.log(e.target); // element that triggered the event
});

// Event delegation – attach one listener to a parent instead of many children
document.querySelector("ul").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") console.log(e.target.textContent);
});

```

Best Practice: Use event delegation for lists/dynamic content to reduce the number of listeners and handle elements added later.

Common Mistake: Attaching individual listeners to many child elements instead of one delegated listener on a parent; forgetting preventDefault() on forms causes an unwanted page reload.

ES6+ Features

Definition: Major JavaScript language additions (ES2015 and later) widely expected in interviews.

```
// Arrow functions (no own `this`)
const add = (a, b) => a + b;

// Destructuring
const { name, age } = user;
const [first, second] = arr;

// Spread / rest
const merged = { ...obj1, ...obj2 };
function sum(...nums) { return nums.reduce((a, b) => a + b); }

// Template literals
const msg = `Hello, ${name}!`;

// Default parameters
function greet(name = "Guest") { console.log(name); }

// Optional chaining & nullish coalescing
const city = user?.address?.city ?? "Unknown";

// Classes
class Animal {
  constructor(name) { this.name = name; }
  speak() { console.log(`${this.name} makes a sound`); }
}
```

Interview Tip: Know that arrow functions do **not** have their own `this`/arguments — they inherit `this` lexically from the enclosing scope, which is why they're unsuitable as object methods needing dynamic `this`.

Modules

Definition: ES Modules (`import/export`) allow splitting code into reusable files with explicit dependencies, each module having its own scope.

```
// math.js
export const add = (a, b) => a + b;
export default function multiply(a, b) { return a * b; }

// main.js
import multiply, { add } from "./math.js";
```

Best Practice: Prefer named exports for multiple utilities (better tooling/autocomplete, tree-shakeable); use default export for a single primary export per file.

Common Mistake: Mixing CommonJS (`require/module.exports`) and ES Modules (`import/export`) without proper build/transpilation configuration.

React

Components

Definition: Reusable, self-contained UI building blocks — typically written as functions returning JSX.

```
function Greeting({ name }) {  
  return <h1>Hello, {name}!</h1>;  
}
```

Best Practice: Keep components small and single-purpose; split large components into smaller, composable ones.

Common Mistake: Mixing multiple unrelated responsibilities in one giant component (“god component”), making it hard to test/reuse.

Props

Definition: Read-only inputs passed from parent to child components, used to configure rendering/behavior.

```
function Button({ label, onClick }) {  
  return <button onClick={onClick}>{label}</button>;  
}  
<Button label="Submit" onClick={() => console.log("clicked")} />
```

Best Practice: Treat props as immutable inside the child; use defaultProps or default parameters for optional props; validate prop shapes with TypeScript or PropTypes.

Common Mistake: Mutating props directly inside a component — always derive local state or pass a callback to the parent instead.

State

Definition: Mutable data local to a component that triggers a re-render when updated (managed via useState/useReducer in function components).

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count + 1)}>{count}</button>;  
}
```

Best Practice: Keep state minimal — derive computed values instead of storing them redundantly; lift state up to the closest common ancestor when shared between siblings.

Common Mistake: Directly mutating state (state.push(x)) instead of creating a new reference (setState([...state, x])) — React relies on reference equality to detect changes.

Hooks

Definition: Functions (useState, useEffect, useContext, etc.) that let function components use state and lifecycle features without classes.

Rules of Hooks: Only call hooks at the top level (not inside loops/conditions/nested functions); only call hooks from React function components or custom hooks.

```
function useToggle(initial = false) { // custom hook
  const [value, setValue] = useState(initial);
  const toggle = () => setValue(v => !v);
  return [value, toggle];
}
```

Common Mistake: Calling hooks conditionally (if (x) { useState() }) — breaks React's hook-order tracking between renders.

useState

Definition: Hook that adds local state to a function component, returning [value, setter].

```
const [items, setItems] = useState([]);

// Functional update – safe when new state depends on previous state
setItems(prevItems => [...prevItems, newItem]);
```

Best Practice: Use the functional updater form (setState(prev => ...)) when the new state depends on the previous state, especially in async callbacks, to avoid stale closures.

Common Mistake: Calling setState multiple times in the same handler expecting each call to see the latest value immediately — state updates are batched and asynchronous.

useEffect

Definition: Hook for running side effects (data fetching, subscriptions, manual DOM manipulation) after render, with an optional cleanup function and dependency array.

```
useEffect(() => {
  const timer = setInterval(() => console.log("tick"), 1000);
  return () => clearInterval(timer); // cleanup on unmount/re-run
}, []); // empty array = run once on mount

useEffect(() => {
  fetchData(id);
}, [id]); // re-run whenever `id` changes
```

Dependency array behavior: - Omitted → runs after every render. - [] → runs once on mount (and cleanup on unmount). - [dep1, dep2] → runs when any listed dependency changes.

Best Practice: Always include every reactive value used inside the effect in the dependency array (use the exhaustive-deps ESLint rule); return a cleanup function for subscriptions/timers/listeners.

Common Mistake: Omitting dependencies to “avoid re-runs,” causing stale closures/bugs; missing cleanup causing memory leaks (e.g., updating state after unmount).

Routing

Definition: Client-side navigation between views without full page reloads, typically via **React Router**.

```
import { BrowserRouter, Routes, Route, Link, useParams } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Link to="/about">About</Link>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/users/:id" element={<UserProfile />} />
      </Routes>
    </BrowserRouter>
  );
}

function UserProfile() {
  const { id } = useParams();
  return <p>User {id}</p>;
}
```

Best Practice: Use lazy loading (React.lazy + Suspense) for route-level code splitting to reduce initial bundle size.

Common Mistake: Using <a href> instead of <Link> for internal navigation, causing a full page reload instead of client-side routing.

Context API

Definition: A way to pass data through the component tree without manually threading props at every level ("prop drilling").

```
const ThemeContext = createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <div className={theme}>Toolbar</div>;
}
```

Best Practice: Use Context for low-frequency global data (theme, auth, locale); for high-frequency updates, combine with state management libraries or split contexts to avoid unnecessary re-renders.

Common Mistake: Overusing Context as a full state-management replacement — every consumer re-renders when the context value changes, which can hurt performance if the value updates often.

Lifecycle

Definition: The phases a component goes through: **Mounting → Updating → Unmounting**. In function components, useEffect covers all these phases via its dependency array and cleanup function.

Class Method	Hook Equivalent
componentDidMount	useEffect(() => {}, [])
componentDidUpdate	useEffect(() => {}, [deps])
componentWillUnmount	cleanup function returned from useEffect

Interview Tip: Be ready to map class lifecycle methods to hooks — a very common conceptual bridge question for candidates coming from class components.

Performance Optimization

Definition: Techniques to minimize unnecessary re-renders and computation in React apps.

```
const MemoizedChild = React.memo(Child); // skip re-render if props unchanged

const expensiveValue = useMemo(() => computeHeavy(a, b), [a, b]); // memoize computed value

const handleClick = useCallback(() => doSomething(id), [id]); // memoize function reference
```

Best Practices: Use React.memo for pure components receiving the same props often; use-Memo/useCallback to stabilize references passed to memoized children; use keys correctly in lists (key={item.id}, never array index for dynamic lists); virtualize long lists (e.g., react-window).

Common Mistake: Using array index as key in dynamic lists (causes incorrect reconciliation on re-order/insert); overusing useMemo/useCallback everywhere, adding complexity without measurable benefit.

Browser & Web APIs

Local Storage

Definition: Persistent key-value storage (~5-10MB) in the browser that survives page reloads and browser restarts, scoped per origin.

```
localStorage.setItem("token", "abc123");
const token = localStorage.getItem("token");
localStorage.removeItem("token");
```

Best Practice: Never store sensitive data (tokens, passwords) in localStorage — it's accessible via JavaScript and vulnerable to XSS.

Common Mistake: Storing large or sensitive data; forgetting localStorage only stores strings (must JSON.stringify/JSON.parse for objects).

Session Storage

Definition: Like localStorage but scoped to a single browser tab session — cleared when the tab is closed.

```
sessionStorage.setItem("formDraft", JSON.stringify(data));
```

Interview Tip: Compare localStorage (persists indefinitely, shared across tabs of the same origin) vs sessionStorage (per-tab, cleared on tab close) vs cookies (sent with every HTTP request, smaller size ~4KB).

Cookies

Definition: Small pieces of data (~4KB) stored by the browser and automatically sent with every HTTP request to the matching domain.

```
document.cookie = "user=John; max-age=3600; Secure; SameSite=Strict";
```

Key attributes: HttpOnly (inaccessible to JS, mitigates XSS), Secure (HTTPS only), SameSite (Strict/Lax/None, mitigates CSRF).

Best Practice: Use HttpOnly + Secure cookies for auth tokens (set by server) rather than storing tokens in localStorage.

Common Mistake: Storing sensitive session tokens in a JS-accessible cookie or localStorage instead of an HttpOnly cookie.

CORS

Definition: Cross-Origin Resource Sharing — a browser security mechanism that restricts web pages from making requests to a different origin (domain/protocol/port) unless the server explicitly allows it via response headers.

Access-Control-Allow-Origin: https://example.com

Access-Control-Allow-Methods: GET, POST

Access-Control-Allow-Headers: Content-Type, Authorization

Key point: CORS is enforced by the **browser**, not the server — the server must opt in via headers; preflight OPTIONS requests are sent automatically for “non-simple” requests (custom headers, non-GET/POST methods, JSON content-type).

Best Practice: Configure CORS on the server to allow only trusted origins, not * (wildcard), especially when credentials are involved.

Common Mistake: Assuming CORS errors mean the request never reached the server — the request often *does* reach the server, but the browser blocks the *response* from being read by the client script.

HTTP Methods

Definition: Standard verbs indicating the desired action on a resource.

Method	Purpose	Idempotent	Body
GET	retrieve a resource	Yes	No
POST	create a resource	No	Yes
PUT	replace a resource entirely	Yes	Yes
PATCH	partially update a resource	No*	Yes
DELETE	remove a resource	Yes	Optional

Best Practice: Use the semantically correct method for RESTful APIs; understand status codes (200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error).

Common Mistake: Using GET for state-changing operations (breaks idempotency assumptions, cacheable by browsers/proxies unexpectedly); confusing 401 (not authenticated) with 403 (authenticated but not authorized).

Best Practices Summary

- Write semantic, accessible HTML by default.
 - Use box-sizing: border-box globally; mobile-first responsive design.
 - Prefer const/let over var; use async/await with proper error handling.
 - Keep React components small, memoize selectively, use stable keys.
 - Never store sensitive data in localStorage; prefer HttpOnly cookies for tokens.
 - Optimize images (loading="lazy", proper formats), minimize render-blocking resources.
 - Use ESLint/Prettier and consistent code style across the team.
-

Performance

Key metrics (Core Web Vitals): **LCP** (Largest Contentful Paint — loading speed), **INP/FID** (interactivity responsiveness), **CLS** (Cumulative Layout Shift — visual stability).

Techniques: code splitting/lazy loading, image optimization, caching (HTTP cache headers, CDN), minimizing bundle size (tree-shaking), debouncing/throttling expensive event handlers, virtualizing long lists.

```
// Debounce example
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
```

Common Mistake: Shipping unoptimized images/fonts; not code-splitting large bundles; triggering layout thrashing with synchronous read/write DOM operations in loops.

Accessibility (General)

Key practices: semantic HTML, sufficient color contrast (WCAG AA: 4.5:1 for normal text), keyboard navigability, visible focus indicators, alt text, ARIA only when necessary, form labels.

Testing tools: Lighthouse, axe DevTools, screen readers (NVDA, VoiceOver), keyboard-only navigation testing.

Common Mistake: Treating accessibility as an afterthought instead of building it in from the start; relying only on automated tools (they catch ~30-40% of issues — manual testing is essential).

Common Interview Question Concepts

- Explain the event loop with a code trace (setTimeout vs Promise).
 - var vs let vs const, and hoisting/TDZ behavior.
 - Difference between == and === (type coercion vs strict equality).
 - Explain closures with a practical example (counter, memoization).
 - CSS specificity and the cascade.
 - Flexbox vs Grid — when to use each.
 - Controlled vs uncontrolled components in React.
 - useEffect dependency array pitfalls.
 - REST API design and correct HTTP status codes.
 - CORS — why errors happen and how to fix them.
-

Common Mistakes (Cross-Cutting)

1. Using == instead of ===, causing unexpected type coercion bugs.
 2. Mutating state/props directly instead of creating new references.
 3. Missing keys or using array index as key in dynamic React lists.
 4. Not cleaning up effects/subscriptions/timers, causing memory leaks.
 5. Ignoring accessibility (missing alt, poor contrast, no keyboard support).
 6. Storing sensitive data in localStorage instead of secure cookies.
 7. Overusing !important and deep CSS specificity instead of proper structure.
 8. Blocking the main thread with heavy synchronous computation.
 9. Not handling Promise rejections / missing error boundaries in React.
 10. Premature or excessive optimization (useMemo/useCallback everywhere) without profiling first.
-

End of Knowledge Base.